```
if grep -q "$pattern" "$file"
then
echo "pattern found"
else
echo "pattern not found"
fi
```

```
cnt=$(grep -c "$pattern" "$file")
echo "count no of lines matching pattern:$cnt"
```

1. **Create a directory 'student'. Under the directory 'student', create 3directories 'stu1', 'stu2' and 'stu3'. Next come back to the home directory and list the created folders: 'stu1', 'stu2' and 'stu3.**

```
mkdir student
cd student
mkdir stu1 stu2 stu3
cd ~
ls student
```

This sequence of commands will create a directory named `student`, navigate into it, create three subdirectories named `stu1`, `stu2`, and `stu3`, return to the home directory, and finally list the contents of the `student` directory from the home directory.

2. **Create a file named 'MyFile', containing student details (your details) and save it under the directory 'stu1'. Copy the file to 'stu2' and 'stu3. Remove the file from 'stu1'. Assure that the file is available only in the directories 'stu2' and 'stu3.**

```
echo "Name: [Your Name]" > MyFile
echo "Age: [Your Age]" >> MyFile
echo "University: [Your University]" >> MyFile
echo "Major: [Your Major]" >> MyFile

cp MyFile student/stu1/
cp MyFile student/stu2/
cp MyFile student/stu3/

rm student/stu1/MyFile

ls student/stu1/
ls student/stu2/
ls student/stu3/
```

Replace `[Your Name]`, `[Your Age]`, `[Your University]`, and `[Your Major]` with your actual details. This sequence of commands will create a file named `MyFile` containing your details, copy it to `stu1`, `stu2`, and `stu3`, remove it from `stu1`, and then list the contents of each directory to confirm the file is only available in `stu2` and `stu3`.

3. **Open the file 'My File'. Find out the number of lines, words and characters present in it. Append data related to subject that you have studied and find again the number of lines, words and characters in the resultant file. Make sure that there is an increase in the number of characters.**

   To accomplish this task, you can use the `wc` command to count lines, words, and characters in a file. You can use any text editor to append data to the file. Here's how you can do it in the terminal:

   a. Open the file 'MyFile' (assuming you're using a command-line text editor like nano or vim):

   ```
   nano MyFile
   ```

   b. After adding data related to the subject you studied, save and exit the editor.

   c. Use the `wc` command to count lines, words, and characters in the file:

   ```
   wc MyFile
   ```

   This will output the number of lines, words, and characters in the file 'MyFile'.

   The increase in the number of characters after appending data should be reflected in the output of the `wc` command. If you've added more content, there should be an increase in the character count.

4. **Create two employee files 'Employee1.txt' and 'Employee2.txt'. Find the common things, the differences and accordingly make the changes in the first file. After changes assure that both files are identical.**

   To achieve this, you can use the `diff` command to find the differences between two files and then make changes to one file to match the other. Here's a step-by-step guide:

a. Create two employee files 'Employee1.txt' and 'Employee2.txt' with some content.

b. Use the `diff` command to find the differences between the two files.

c. Make changes to 'Employee1.txt' to match 'Employee2.txt'.

d. Verify that both files are identical.

Here's how you can do it:

```
# Create the files and add some content
echo "Employee 1 Details:" > Employee1.txt
echo "Name: John" >> Employee1.txt
echo "Age: 30" >> Employee1.txt
echo "Position: Manager" >> Employee1.txt

echo "Employee 2 Details:" > Employee2.txt
echo "Name: John" >> Employee2.txt
echo "Age: 30" >> Employee2.txt
echo "Position: Senior Manager" >> Employee2.txt

# Use diff to find the differences
diff Employee1.txt Employee2.txt

# Make changes to Employee1.txt to match Employee2.txt
sed -i 's/Position: Manager/Position: Senior Manager/' E
mployee1.txt

# Verify that both files are identical
diff Employee1.txt Employee2.txt
```

This sequence of commands creates two files with employee details, finds the differences between them, makes changes to 'Employee1.txt' to match 'Employee2.txt', and then verifies that both files are identical using the `diff` command again.

5. **Create a file 'Books.txt' containing book details like Book_id , Book_Name, Publisher_Name, Author_Name, Date_of_publication, Book_Type (Technical/Non_technical), Book_subject (OS, C_Language,**

FCO, Cyber_Forensics, SAD, DBMS etc.), No of books and Description. Do the following operations using the 'grep' command:

a. **Search for the 'DBMS' books**
b. **Count number of books which are technical/TecHnical.**
c. **Search for book names containing where the author name contains the word "Acharya" or "Sharma" or "Pandit".**
d. **Look for the records with publisher "Mcgraw" or "mcgraw".**
e. **Find all the nontechnical books along with their line numbers.**
f. **Find for the records that doesn't contain the word 'is'.**
g. **Search for the books where the Book_id begins with a 2.**

To perform these operations using the `grep` command, you can use various options provided by `grep` for pattern matching. Here's how you can do each operation:

a) Search for the 'DBMS' books:

```
grep 'DBMS' Books.txt
```

b) Count the number of books which are technical/TecHnical:

```
grep -i 'Technical' Books.txt | wc -l
```

c) Search for book names containing where the author name contains the word "Acharya" or "Sharma" or "Pandit":

a.
```
grep -E 'Acharya|Sharma|Pandit' Books.txt | grep 'Boo
k_Name'
```

d) Look for the records with publisher "Mcgraw" or "mcgraw":

```
grep -i 'Mcgraw' Books.txt
```

e) Find all the non-technical books along with their line numbers:

```
grep -n -v 'Technical' Books.txt
```

f) Find the records that don't contain the word 'is':

```
grep -v 'is' Books.txt
```

g) Search for the books where the Book_id begins with a 2:

```
grep '^2' Books.txt
```

These `grep` commands should help you perform the operations specified on the 'Books.txt' file as described. Adjustments can be made based on the actual structure of the file and specific requirements.

6. **Create a student file, containing the marks of students in the ascending order for mathematics subject. Find out the first five toppers as well as two students with the least marks in the said subject.**

   To accomplish this task, you can follow these steps:

   a. Create a student file with marks for the mathematics subject in ascending order.

   b. Use the `head` command to find the first five toppers.

   c. Use the `tail` command to find the two students with the least marks.

   Here's how you can do it:

```
# Create the student file with marks for mathematics sub
ject
echo "Student Marks for Mathematics:" > student_marks.tx
t
echo "John 75" >> student_marks.txt
echo "Alice 80" >> student_marks.txt
echo "Bob 85" >> student_marks.txt
echo "Charlie 90" >> student_marks.txt
echo "Diana 95" >> student_marks.txt
echo "Eva 60" >> student_marks.txt
echo "Frank 65" >> student_marks.txt
echo "Grace 70" >> student_marks.txt
echo "Henry 55" >> student_marks.txt
echo "Isabella 50" >> student_marks.txt
```

```
# Sort the file based on marks (second column) in ascend
ing order
sort -k2n student_marks.txt > sorted_student_marks.txt

# Find the first five toppers
echo "Top Five Toppers for Mathematics:"
head -n 5 sorted_student_marks.txt

# Find the two students with the least marks
echo "Two Students with the Least Marks for Mathematic
s:"
tail -n 2 sorted_student_marks.txt
```

This script will create a file named `student_marks.txt` containing student names and their marks for mathematics, then sort the file based on marks in ascending order. Finally, it will display the first five toppers and the two students with the least marks. Adjust the file contents and names as needed.

7. **Create a file "fruits.txt" containing different fruits and do the following Operations. Enter minimum 10 fruits.**
   a. **Sort the file in the ascending order.**
   b. **Sort the file in the reverse order.**
   c. **Sort the file and remove the duplicates.**
   d. **Edit the file by adding another filed for the "stock" by using some delimiter for the fields. Sort the file on the second column.**
   e. **Find the top 3 fruits after sorting.**
   f. **Find the last 3 fruits after sorting.**

Here's how you can perform each of the operations using various command-line tools such as `sort`, `uniq`, and `head`/`tail`:

a. Sort the file in ascending order:

```
sort fruits.txt
```

b. Sort the file in reverse order:

```
sort -r fruits.txt
```

c. Sort the file and remove duplicates:

```
sort -u fruits.txt
```

d. Edit the file by adding another field for the "stock" using some delimiter and sort the file on the second column:

```
# Assuming you want to add stock details separated by
a comma
awk '{print $0 ",stock"}' fruits.txt | sort -t',' -k2
```

e. Find the top 3 fruits after sorting:

```
sort fruits.txt | head -n 3
```

f. Find the last 3 fruits after sorting:

```
sort fruits.txt | tail -n 3
```

Make sure to replace `fruits.txt` with the actual filename containing the list of fruits. Adjustments can be made based on your specific requirements or file format.

8. **Write a shell script to print the date displaying: Day_of_the_week, Month_Name and Year, with a space delimiter.**

Certainly! You can achieve this using the `date` command with appropriate formatting options in a shell script. Here's a script to do that:

```
#!/bin/bash

# Get the current date in the specified format
formatted_date=$(date "+%A %B %Y")

# Print the formatted date
echo "$formatted_date"
```

Save the above script in a file, for example, `print_date.sh`. Make it executable using the following command:

```
chmod +x print_date.sh
```

Now you can run the script using `./print_date.sh`, and it will print the current date in the format "Day_of_the_week Month_Name Year", separated by spaces.

9. **Write a shell script to find the total number of files available in the home directory.**

   You can achieve this by using the `find` command to list all files in the home directory and then count them. Here's a shell script to accomplish that:

```
#!/bin/bash

# Use find command to list all files in the home directo
ry and count them
total_files=$(find "$HOME" -type f | wc -l)

# Print the total number of files
echo "Total number of files in the home directory: $tota
l_files"
```

   Save this script in a file, for example, `count_files.sh`, then make it executable using the following command:

```
chmod +x count_files.sh
```

   Now you can run the script using `./count_files.sh`, and it will display the total number of files in your home directory.

10. **Write a shell script to print the command line arguments as well as the total number of arguments.**

   You can achieve this by utilizing special variables provided by the shell. Here's a shell script that prints the command line arguments along with the total number of arguments:

```
#!/bin/bash

# Print all command line arguments
echo "Command line arguments:"
echo "$@"

# Print the total number of arguments
total_arguments=$#
echo "Total number of arguments: $total_arguments"
```

Save this script in a file, for example, `print_arguments.sh`, then make it executable using the following command:

```
chmod +x print_arguments.sh
```

Now you can run the script with any number of arguments, for example:

```
./print_arguments.sh arg1 arg2 arg3
```

And it will print the provided arguments along with the total number of arguments.

11. **Write a shell script to enter your First name and Second name and display your full name.**

    You can create a simple shell script to prompt the user to enter their first and last names, then display their full name. Here's how you can do it:

```
#!/bin/bash

# Prompt the user to enter their first name
echo -n "Enter your First name: "
read first_name

# Prompt the user to enter their last name
echo -n "Enter your Last name: "
read last_name
```

```
# Display the full name
echo "Your full name is: $first_name $last_name"
```

Save this script in a file, for example, `full_name.sh`, then make it executable using the following command:

```
chmod +x full_name.sh
```

Now you can run the script using `./full_name.sh`, it will prompt you to enter your first and last names, and then display your full name.

12. **Write a shell script to display all the pizzas you like using command line arguments.**

    You can create a shell script that accepts command-line arguments representing the pizzas you like and then displays them. Here's a simple example:

```
my_favorite_pizzas.sh
#!/bin/bash

# Check if there are no command line arguments
if [ "$#" -eq 0 ]; then
    echo "Usage: $0 [pizza1] [pizza2] ..."
    exit 1
fi

# Print the pizzas you like
echo "Pizzas I like:"
for pizza in "$@"; do
    echo "- $pizza"
done
```

Save this script in a file, for example, , then make it executable using the following command:

```
chmod +x my_favorite_pizzas.sh
```

Now you can run the script with the names of your favorite pizzas as command line arguments, like this:

```
./my_favorite_pizzas.sh Margherita Pepperoni Hawaiian
```

It will display the pizzas you like, one per line. If you run it without any arguments, it will display a usage message.

13. Write a shell script to enter your First name and Second name and display your full name.

name.sh

```
echo "Enter your first name: "
read fname
echo "Enter your second name: "
read lname
echo "Full name: $fname $lname"
```

```
~$ vi name.sh
~$ chmod +x name.sh
~$ ./name.sh
Enter your first name:
Sam
Enter your second name:
Fisher
Full name: Sam Fisher
```

14. Write a shell script to display all the pizzas you like using command line arguments.

pizza.sh

```
echo "My favorite pizza are: $@"
```

```
~$ vi pizza.sh
~$ chmod +x pizza.sh
~$ ./pizza.sh Mushroom Farmhouse Margerita
My favorite pizza are: Mushroom Farmhouse Margerita
```

15. Write a report on the command line arguments. Illustrate their use with some examples.

Command line arguments are parameters that are passed into a program when it is executed in a Command Line Interface (CLI). These arguments provide additional information or options to the program being executed. It allows user to customize the behavior of programs without modifying their source code.

Syntax: program name [option 1] [option 2] [option 3]

Use of command line arguments:

1. **Providing input data:** They can be used to provide input data to a program.

   Example: A file or string name to be processed.

2. **Setting program options:** They can be used to specify various options or flags that control the behaviour of the program. These options can enable or disable certain features, specify output formats ,or set parameters.

3. **Passing parameters:** They can also be used to pass parameters to a program, which can influence its execution logic.

Examples:

1. Ls command:

```
~$ ls -l
total 3
-rw-r--r-- 1 user user   0 Apr  7 10:59 linux
-rwxr-xr-x 1 user user 117 Apr  7 11:10 name.sh
-rwxr-xr-x 1 user user  33 Apr  7 11:33 pizza.sh
drwxr-xr-x 2 user user   2 Apr  7 11:01 student
```

2. cat command

```
~$ cat >file1
Mango
Orange
Apple
Banana
```

3. grep command

```
~$ grep Mango file1
Mango
```

16. Write a shell script to check for the existence of a file.

```
~$ vi exists.sh
~$ chmod +x exists.sh
~$ ./exists.sh
Enter file name:
pizza.sh
File pizza.sh exists!
~$ ./exists.sh
Enter file name:
pizza
File pizza doesn't exists!
```

17. Write a shell script to:

a. Enter two file names in the command line

b. Check that correct number of arguments have been entered through the command line

c. Read and display the file contents

d. Compare the files. Also search for a particular pattern in the first file.

filecommands.sh

```
# Check if correct number of arguments are provided
#-ne is not equal command
if [ "$#" -ne 2 ]; then
echo "Files to be used: $0 <file1> <file2>"
exit
fi

# Store file names from command line arguments
file1=$1
file2=$2

# Check if both files exist
if [ ! -e "$file1" ]
then
echo "File '$file1' does not exist!"
exit
fi

if [ ! -e "$file2" ]
then
echo "File '$file2' does not exist!"
exit
fi

# Display file contents
echo "Contents of $file1: "
cat "$file1"
echo "Contents of $file2: "
cat "$file2"

# Compare both the files
# -s command checks if file exists and has non-zero size
if cmp -s "$file1" "$file2"
then
echo "Files '$file1' and '$file2' are identical."
else
echo "Files '$file1' and '$file2' are different."
fi
```

```
# Search for a particular pattern in the first file
echo "Enter a pattern to search in $file1: "
read pattern
if grep -q "$pattern" "$file1"
then
echo "Pattern '$pattern' found in $file1."
else
echo "Pattern '$pattern' not found in $file1."
fi

```

```
~$ vi filecommands.sh
~$ chmod +x filecommands.sh
~$ ./filecommands.sh file1 file2
Contents of file1:
Mango
Orange
Apple
Banana
Contents of file2:
Raspberry
Dragonfruit
Blueberry
Kiwi
Files 'file1' and 'file2' are different.
Enter a pattern to search in file1:
Banana
Pattern 'Banana' found in file1.
```

18. Write a shell script to create a directory. Create a file under this directory and check that the file is executable or not.

createdir.sh

```
# Create a directory
echo "Enter the directory name: "
read dirname

mkdir "$dirname"

if [ $# -ne 0 ]
then
echo "Failed to create directory!"
exit
fi

echo "Directory "$dirname" created successfully!"

# Create a file under the directory
echo "Enter file name: "
read filename

#touch command used to create new empty file.
touch "$dirname/$filename"

if [ $# -ne 0 ]
then
echo "Failed to create file!"
exit
fi

echo "File '$filename' created under directory "$dirname"."

# Check if the file is executable
#-x is execute
if [ -x "$dirname/$filename" ]
then
echo "File "$filename" is executable!"
else
echo "File "$filename" is not executable!"
fi
```

```
~$ vi createdir.sh
~$ chmod +x createdir.sh
~$ ./createdir.sh
Enter the directory name:
dirfile
Directory dirfile created successfully!
Enter file name:
main.txt
File 'main.txt' created under directory dirfile.
File main.txt is not executable!
```

19. Write a shell script to find the number of words in two files, compare them and print appropriate messages accordingly. At first check for the number of arguments entered at the command line and check that the files are existing.

count.sh

```
# Check if correct number of arguments are provided
if [ "$#" -ne 2 ]
then
echo "Files to be used: $0 <file1> <file2>"
exit
fi
file1=$1
file2=$2

# Check if both files exist
if [ ! -e "$file1" ]
then
echo "File '$file1' does not exist!"
exit
fi

if [ ! -e "$file2" ]
then
echo "File '$file2' does not exist!"
exit
fi

# Function to count words in a file
#local keyword is for local variable scope
word_count()
{
    local file="$1"
    local count=$(wc -w < "$file")
    echo "$count"
}

# Count words in both files
words_file1=$(word_count "$file1")
words_file2=$(word_count "$file2")

```

```
# Compare word counts
if [ "$words_file1" -eq "$words_file2" ]
then
echo "Both files have the same number of words: $words_file1 words."
elif [ "$words_file1" -gt "$words_file2" ]
then
echo "File '$file1' has more words "$words_file1" than file '$file2' "$words_file2"!"
else
echo "File '$file2' has more words "$words_file2" than file '$file1' "$words_file1"!"
fi
```

```
~$ vi count.sh
~$ chmod +x count.sh
~$ ./count.sh file1 file2
Both files have the same number of words: 4 words.
~$ ./count.sh file1 file7
File 'file7' does not exist!
~$ ./count.sh file1 pizza.sh
File 'pizza.sh' has more words 6 than file 'file1' 4!
~$ ./count.sh
Files to be used: ./count.sh <file1> <file2>
```

20. Write a shell script to accept number and check the number is even or odd.

oddeven.sh

```
echo "Enter a number:"
read num

if [ $((num % 2)) -eq 0 ]
then
echo "$num is EVEN!"
else
echo "$num is ODD!"
fi
```

```
~$ vi oddeven.sh
~$ chmod +x oddeven.sh
~$ ./oddeven.sh
Enter a number:
4
4 is EVEN!
~$ ./oddeven.sh
Enter a number:
2425
2425 is ODD!
```

21. Write a shell script for adding two numbers.

add.sh

```
echo "Enter the first number: "
read num1

echo "Enter the second number: "
read num2

sum=$((num1 + num2))

echo "Sum: $sum"
```

```
~$ vi add.sh
~$ chmod +x add.sh
~$ ./add.sh
Enter the first number:
10
Enter the second number:
200
Sum: 210
```

22. Shell script for checking that a file has been copied successfully and to print the appropriate messages accordingly. (Use command line arguments)

copy.sh

```
# Check if correct number of arguments are provided
if [ "$#" -ne 2 ]
then
echo "Files to be used: $0 <source_file> <destination_file>"
exit
fi

source_file=$1
destination_file=$2

# Check if source file exists
if [ ! -e "$source_file" ]
then
echo "Source file does not exist!"
exit
fi

# Copy the file
cp "$source_file" "$destination_file"

# Check if destination file exists (to determine if the copy was successful)
if [ -e "$destination_file" ]
then
echo "File has been successfully copied!"
else
echo "File was not copied successfully!"
fi
```

```
~$ vi copy.sh
~$ chmod +x copy.sh
~$ ./copy.sh file1 file2
File has been successfully copied!
~$ cat file1
Mango
Orange
Apple
Banana
~$ cat file2
Mango
Orange
Apple
Banana
```

23. Check that a file is having read, write, and execute permissions. Print appropriate messages. Also check for the length of the filename, if the length is zero then print the message "Improper filename."

file.sh

```
# Check if correct number of arguments are provided
if [ "$#" -ne 1 ]
then
echo "Usage: $0 <filename>"
exit
fi

filename=$1

# Check if filename length is zero
if [ ${#filename} -eq 0 ]
then
echo "Improper filename!"
exit
fi

# Check if file exists
if [ ! -e "$filename" ]
then
echo "File name doesn't exists!"
exit
fi

# Check permissions
if [ -r "$filename" ] && [ -w "$filename" ] && [ -x "$filename" ]
then
echo "File has read, write, and execute permissions!"
elif [ -r "$filename" ] && [ -w "$filename" ]
then
echo "File only has read and write permissions!"
elif [ -r "$filename" ] && [ -x "$filename" ]
then
echo "File only has read and execute permissions!"
elif [ -w "$filename" ] && [ -x "$filename" ]
then
echo "File only has write and execute permissions!"
elif [ -r "$filename" ]
then
echo "File only has read permission!"
elif [ -w "$filename" ]
then
echo "File only has write permission!"
elif [ -x "$filename" ]
then
echo "File only has execute permission!"
else
echo "File lacks read, write, and execute!"
fi
```

```
~$ vi file.sh
~$ chmod +x file.sh
~$ ./file.sh file1
File only has read and write permissions!
~$ ./file.sh pizza.sh
File has read, write, and execute permissions!
```

24. Using command line arguments enter marks of students for 5 subjects, and print the appropriate classes. Following are the criteria for printing messages:

a. If percentage is more than 60, then print "First class"

b. If percentage is between 50 and 60, then print "Second class"

c. If percentage is between 40 and 50, then print "Third class"

If percentage is less than 40, then print "Fail"

grade.sh

```
# Check if correct number of arguments are provided
if [ "$#" -ne 5 ]
then
echo "Useage: $0 <subject1_marks> <subject2_marks> <subject3_marks> <subject4_marks> <subject5_marks>"
exit 1
fi

# Store marks from command line arguments
marks1=$1
marks2=$2
marks3=$3
marks4=$4
marks5=$5

# Calculate total marks
total_marks=$((marks1 + marks2 + marks3 + marks4 + marks5))

# Calculate percentage
percentage=$(($total_marks / 5))

# Determine class based on percentage
if [ $percentage -gt 60 ]
then
echo "Percentage: $percentage  Class: First class"
elif [ $percentage -ge 50 ] && [ $percentage -le 60 ]
then
echo "Percentage: $percentage  Class: Second class"
elif [ $percentage -ge 40 ] && [ $percentage -le 50 ]
then
echo "Percentage: $percentage  Class: Third class"
else
echo "Percentage: $percentage  Class: Fail"
fi
```

```
~$ vi grade.sh
~$ chmod +x grade.sh
~$ ./grade.sh 59 78 99 60 54
Percentage: 70  Class: First class
```

25. Create a menu driven program to

1) To create a directory

2) To check for the existence of file

3) For displaying the contents of a file

4) To move a file. The default case is to print the message "Not a correct choice:"

menu.sh

```
while true
do
# Display menu options
echo "Menu
1) Create a directory
2) Check for the existence of a file
3) Display the contents of a file
4) Move a file
5) Exit

Enter your choice: "

read choice
case $choice in

# Create a directory
1)echo "Enter the directory name: "
read dirname
mkdir "$dirname"
if [ $# -eq 0 ]
then
echo "Directory created successfully!"
else
echo "Failed to create directory!"
fi;;

# Check for the existence of a file
2)echo "Enter the filename to check: "
read filename
if [ -e "$filename" ]
then
echo "File exists!"
else
echo "File does not exist!"
fi;;

# Display the contents of a file
3)echo "Enter the filename to display: "
read filename
if [ -e "$filename" ]
then
cat "$filename"
else
echo "File does not exist!"
fi;;

# Move a file
4)  echo "Enter the source file: "
read sourcefile
echo "Enter the destination directory: "
read destdir
mv "$sourcefile" "$destdir"
if [ $# -eq 0 ]
then
echo "File moved successfully!"
else
echo "Failed to move file."
fi;;

# Exit the program
5)echo "Exiting program."
exit 0;;

*)echo "Not a correct choice!";;
esac
done
```

```
~$ vi menu.sh
~$ chmod +x menu.sh
~$ ./menu.sh
Menu
1) Create a directory
2) Check for the existence of a file
3) Display the contents of a file
4) Move a file
5) Exit

Enter your choice:
1
Enter the directory name:
menufiles
Directory created successfully!
Menu
1) Create a directory
2) Check for the existence of a file
3) Display the contents of a file
4) Move a file
5) Exit

Enter your choice:
2
Enter the filename to check:
file1
File exists!
Menu
1) Create a directory
2) Check for the existence of a file
3) Display the contents of a file
4) Move a file
5) Exit
```

```
Enter your choice:
3
Enter the filename to display:
file2
Mango
Orange
Apple
Banana
Menu
1) Create a directory
2) Check for the existence of a file
3) Display the contents of a file
4) Move a file
5) Exit

Enter your choice:
4
Enter the source file:
file1
Enter the destination directory:
menufiles
File moved successfully!
Menu
1) Create a directory
2) Check for the existence of a file
3) Display the contents of a file
4) Move a file
5) Exit

Enter your choice:
5
Exiting program.
```

26) Write a shell script for reading a given file.

```
  Ans::  # Check if a filename was provided as an argument
if [ $# -eq 0 ]; then
  echo "Usage: $0 <filename>"
  exit 1
fi

# Check if the provided file exists
if [ ! -f "$1" ]; then
  echo "Error: File '$1' not found."
  exit 1
fi

# Read and display the contents of the file
echo "Contents of '$1':"
cat "$1"

Save this script in a file, e.g., read_file.sh, and make it executable
with the command chmod +x read_file.sh.
Then you can run it with the filename you want to read as an argument,
like ./read_file.sh example.txt.
```

27) Print values within a range using while loop.

```
# Define the range start and end values
start=1
end=10

# Initialize a counter variable
counter=$start

# Use a while loop to print values within the range
while [ $counter -le $end ]; do
  echo $counter
  ((counter++))
done
```

28) Write a shell script to display the name of the country and the language being used with help of a case structure.

```
# Function to display country name and language
display_country_language() {
    case "$1" in
        "USA")
            echo "Country: United States of America"
            echo "Language: English"
            ;;
        "France")
            echo "Country: France"
            echo "Language: French"
            ;;
```

```
        "Japan")
            echo "Country: Japan"
            echo "Language: Japanese"
            ;;
        "Germany")
            echo "Country: Germany"
            echo "Language: German"
            ;;
        *)
            echo "Unknown country!"
            ;;
    esac
}

# Check if an argument was provided
if [ $# -eq 0 ]; then
    echo "Usage: $0 <country>"
    exit 1
fi

# Call the function with the provided country name
display_country_language "$1"
```

29) Write a shell script to create a menu for entering the choice of colour andprint the appropriate message according the entered colour.

```
#!/bin/bash

# Function to display the color menu and print the appropriate message
color_menu() {
    echo "Choose a color:"
    echo "1. Red"
    echo "2. Blue"
    echo "3. Green"
    echo "4. Yellow"
    echo "5. Exit"

    read -p "Enter your choice (1-5): " choice

    case $choice in
        1)
            echo "You chose Red. It symbolizes passion and energy."
            ;;
        2)
            echo "You chose Blue. It represents calmness and
tranquility."
            ;;
        3)
            echo "You chose Green. It signifies growth and harmony."
            ;;
        4)
```

```
            echo "You chose Yellow. It symbolizes happiness and
positivity."
            ;;
        5)
            echo "Exiting program."
            exit 0
            ;;
        *)
            echo "Invalid choice. Please enter a number from 1 to 5."
            ;;
    esac
}

# Main loop to display the color menu repeatedly
while true; do
    color_menu
done
```

30) Write a shell script to print a range of values using until loop.

```
# Define the range start and end values
start=1
end=10

# Initialize a counter variable
counter=$start

# Use an until loop to print values within the range
until [ $counter -gt $end ]; do
  echo $counter
  ((counter++))
done
```


31) Write a shell script to rename a set of files.

```
# Check if a directory name was provided as an argument
if [ $# -eq 0 ]; then
  echo "Usage: $0 <directory>"
  exit 1
fi

# Check if the provided directory exists
if [ ! -d "$1" ]; then
  echo "Error: Directory '$1' not found."
  exit 1
fi

# Change to the specified directory
cd "$1" || exit

# List the files in the directory
```

```
#!/bin/bash

i=1
for f in *; do
  mv "$f" "new_$i.${f##*.}"
  ((i++))
done

echo "Renamed all files in the current
directory."
```

```
files=$(ls)

# Loop through each file and rename it
for file in $files; do
  # Rename the file by adding a prefix "new_" to the original filename
  new_name="new_$file"
  mv "$file" "$new_name"
  echo "Renamed $file to $new_name"
done
```

32) Write a script to print the names of your friends using for loop and Command line arguments.

```
# Check if command-line arguments are provided
if [ $# -eq 0 ]; then
  echo "Usage: $0 <friend1> <friend2> <friend3> ..."
  exit 1
fi

# Loop through each command-line argument (friends' names)
for friend in "$@"; do
  echo "Hello, $friend!"
done
```

33) Write a script to print all the sub directories in the current directory.

```
# Check if the current directory is empty
if [ -z "$(ls -A .)" ]; then
  echo "Current directory is empty."
  exit 1
fi

# Loop through each entry in the current directory
for entry in *; do
  # Check if the entry is a directory
  if [ -d "$entry" ]; then
    echo "$entry"
  fi
done
```

```
if [ -z "$(ls -A)" ]
then
   echo "dir empty"
   exit
fi
echo "subdirectories are:"
for sub in *;do
if [ -d "$sub" ]
then
   echo "$sub"
fi
done
```

34) Write a script to check the entered arguments at the command line are directories or files. Print messages accordingly indicating that it is a file or directory. If the argument does not exist then create a directory with the name of the argument supplied.

```
# Check if command-line arguments are provided
if [ $# -eq 0 ]; then
  echo "Usage: $0 <file_or_directory1> <file_or_directory2> ..."
  exit 1
```

```
fi

# Loop through each argument provided
for arg in "$@"; do
  # Check if the argument exists
  if [ -e "$arg" ]; then
    # Check if the argument is a directory
    if [ -d "$arg" ]; then
      echo "$arg is a directory."
    else
      echo "$arg is a file."
    fi
  else
    # Create a directory with the argument name
    mkdir "$arg"
    if [ $? -eq 0 ]; then
      echo "Directory '$arg' created successfully."
    else
      echo "Error creating directory '$arg'."
    fi
  fi
done
```

35) Write a script to calculate gross salary for any number of employeesGross Salary =Basic + HRA + DA.
HRA=10% and DA= 15%.

```
if [ "$#" -eq 0 ]
then
echo "usage:$0 <e1> <e2>"
exit 0
fi
echo "salary entered"
for sal in "$@"; do
hra=$(echo "$sal*10/100")
da=$(echo "$sal*15/100")
gross=$((sal+hra+da))
echo "$gross"
 done
```

```
# Function to calculate gross salary
calculate_gross_salary() {
    # Arguments: $1 = Basic salary
    basic_salary=$1
    hra_percentage=10
    da_percentage=15

    # Calculate HRA and DA
    hra=$(echo "scale=2; $basic_salary * $hra_percentage / 100" | bc)
    da=$(echo "scale=2; $basic_salary * $da_percentage / 100" | bc)

    # Calculate gross salary
    gross_salary=$(echo "scale=2; $basic_salary + $hra + $da" | bc)

    # Print the gross salary
    echo "Basic Salary: $basic_salary"
    echo "HRA: $hra"
    echo "DA: $da"
    echo "Gross Salary: $gross_salary"
}

# Check if command-line arguments are provided
if [ $# -eq 0 ]; then
  echo "Usage: $0 <basic_salary1> <basic_salary2> ..."
```

```
  exit 1
fi

# Loop through each basic salary provided as argument
for salary in "$@"; do
  calculate_gross_salary "$salary"
  echo "============================="
done
```

36) Write a shell script to print the values of $0, $*, $@, $1, $2 and $#.

```
# Print the value of $0 (script name)
echo "Value of \$0 (script name): $0"

# Print the value of $* (all command-line arguments as a single string)
echo "Value of \$* (all arguments as a single string): $*"

# Print the value of $@ (all command-line arguments as separate strings)
echo "Value of \$@ (all arguments as separate strings): $@"

# Print the value of $1 (first command-line argument)
echo "Value of \$1 (first argument): $1"

# Print the value of $2 (second command-line argument)
echo "Value of \$2 (second argument): $2"

# Print the value of $# (number of command-line arguments)
echo "Value of \$# (number of arguments): $#"
```

37) Write a shell script to append contents in an existing file. First check forthe existence of file.

```
# Function to append contents to a file
append_to_file() {
    local filename=$1
    local content=$2

    if [ -f "$filename" ]; then
        echo "$content" >> "$filename"
        echo "Content appended to '$filename'."
    else
        echo "Error: File '$filename' does not exist."
        exit 1
    fi
}

# Check if command-line arguments are provided
if [ $# -ne 2 ]; then
  echo "Usage: $0 <filename> <content>"
  exit 1
fi
```

```
filename=$1
content=$2

# Call the function to append contents to the file
append_to_file "$filename" "$content"
```

38. Write a script to copy the contents of one file into the other. Enter the file names using command line.

```
#-your variable or value that you read
# Check if a filename is provided as an argument
if [ $# -ne 1 ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi

filename=$1

# Check if the file exists
if [ ! -f "$filename" ]; then
    echo "Error: File '$filename' not found."
    exit 1
fi

# Display the contents of the file line by line
echo "Contents of $filename:"
while IFS= read -r line; do
    echo "$line"
done < "$filename"

# Count the number of lines in the file
num_lines=$(wc -l < "$filename")
echo "Number of lines in $filename: $num_lines"

# Prompt for confirmation before deleting the file
read -p "Are you sure you want to delete $filename? (y/n): " confirm
if [ "$confirm" = "y" ]; then
    # Delete the file
    rm "$filename"
    echo "File '$filename' has been deleted."
else
    echo "Deletion aborted."
fi


chmod +x delete_file.sh
./delete_file.sh filename.txt
```

```
if [ "$#" -ne 2 ]
then
echo "not found name"
exit
fi
file1=$1
file2=$2
if  [ ! -f "$file1" ]
then
   echo "file1 does not exist"
   exit
fi
if cp "$file1" "$file2"; then
   echo "file has been copied"
else
   echo "not copied"
fi
```

39. Write a script to copy the contents of one file into the other. Enter the file names using command line.

```
# Check if two filenames are provided as arguments
if [ $# -ne 2 ]; then
    echo "Usage: $0 <source_file> <destination_file>"
    exit 1
fi
source_file=$1
destination_file=$2
```

```
# Check if the source file exists
if [ ! -f "$source_file" ]; then
    echo "Error: Source file '$source_file' not found."
    exit 1
fi
# Copy the contents of source file into destination file
cp "$source_file" "$destination_file"
echo "Contents of '$source_file' have been copied to
'$destination_file'."
chmod +x copy_file_contents.sh
./copy_file_contents.sh source_file.txt destination_file.txt
```

40. Write a script to print the numbers from 1 to 10 by skipping the value of 6 Write both by using infinite while and until loop.

```
# Using while loop
num=1
while [ $num -le 10 ]; do
    if [ $num -eq 6 ]; then
        ((num++))
        continue
    fi
    echo $num
    ((num++))
done
# Using until loop
num=1
until [ $num -gt 10 ]; do
    if [ $num -eq 6 ]; then
        ((num++))
        continue
    fi
    echo $num
    ((num++))
done
```

41. Write a script to read the marks of students and to find the percentage ofa student. Also print their class as below:
a. Distinction if percentage is greater than or equal to 70.
b. First class if percentage is greater than or equal to 60.
c. Second class if percentage is between 50 and 60.
d. Third class if percentage is greater than or equal to 40.
Fail if percentage is less than or equal to 40.

```
# Function to calculate percentage and determine class
calculate_percentage_and_class() {
    marks=$1
    total_marks=$2

    percentage=$(echo "scale=2; ($marks / $total_marks) * 100" | bc)

    echo "Percentage: $percentage%"
```

```
    if (( $(echo "$percentage >= 70" | bc -l) )); then
        echo "Class: Distinction"
    elif (( $(echo "$percentage >= 60" | bc -l) )); then
        echo "Class: First class"
    elif (( $(echo "$percentage >= 50 && $percentage < 60" | bc -l) ));
then
        echo "Class: Second class"
    elif (( $(echo "$percentage >= 40 && $percentage < 50" | bc -l) ));
then
        echo "Class: Third class"
    else
        echo "Class: Fail"
    fi
}

# Read marks from user input
read -p "Enter marks obtained: " marks
read -p "Enter total marks: " total_marks

# Check if marks and total_marks are valid numbers
if ! [[ $marks =~ ^[0-9]+$ ]] || ! [[ $total_marks =~ ^[0-9]+$ ]]; then
    echo "Error: Invalid input. Please enter valid numbers for marks and
total marks."
    exit 1
fi
# Call function to calculate percentage and determine class
calculate_percentage_and_class $marks $total_marks
```

42. Write a shell script to enter the arguments from the command line. Print Message for entering an argument and exit from the program if argumentis supplied, else print Thank you.

```
#-your variable or value that you read
# Check if an argument is supplied
if [ $# -eq 0 ]; then
    echo "Thank you"
    exit 0
fi
# Print message for entering an argument and exit
echo "Argument entered: $1"
exit 0
```

43. Create the following menu:
1-Blue2-Red
3-Yellow4-
Green 5-Orange
Prompt the user to enter the choice of the colour and accordingly printtheir choice.

```
# Function to display the menu
display_menu() {
    echo "Color Menu:"
    echo "1- Blue"
```

```
    echo "2- Red"
    echo "3- Yellow"
    echo "4- Green"
    echo "5- Orange"
}
# Function to print the selected color
print_color() {
    case $1 in
        1) echo "You selected Blue." ;;
        2) echo "You selected Red." ;;
        3) echo "You selected Yellow." ;;
        4) echo "You selected Green." ;;
        5) echo "You selected Orange." ;;
        *) echo "Invalid choice. Please select a number between 1 and 5."
;;
    esac
}
# Main function
main() {
    display_menu
    read -p "Enter your choice (1-5): " choice

    case $choice in
        1|2|3|4|5)  # Check if choice is between 1 and 5
            print_color "$choice"
            ;;
        *)  # If choice is not between 1 and 5
            echo "Invalid choice. Please select a number between 1 and
5."
            ;;
    esac
}
# Call the main function
main
```

44. Write a shell script for an employee file to (create a menu)
a. Count the number of lines matching a given pattern
b. Display the employees whose city is not in Ahmedabad
c. To display the top two records
d. Count the number of words in the file.

```
# Function to count the number of lines matching a given pattern
count_matching_lines() {
    read -p "Enter the pattern to search: " pattern
    count=$(grep -c "$pattern" employee_file.txt)
    echo "Number of lines matching '$pattern': $count"
}
# Function to display employees whose city is not Ahmedabad
display_non_ahmedabad() {
    echo "Employees whose city is not Ahmedabad:"
    grep -v "Ahmedabad" employee_file.txt
}
# Function to display the top two records
```

```
if [ "$#" -ne 1 ]
then
   echo "usage:$0 <file>"
   exit
fi
file=$1
while true
do
echo "menu:
1)count no of lines matching a pattern
2)display employees whose city is not
in ahmedabad
3)display top 2 records
4)count no of words in file
enter ur choice:"
read choice
case $choice in
1)echo -n "enter the pattern to
search:"
read pattern
cnt=$(grep -c "$pattern" "$file")
echo "$cnt";;
2)grep -v "Ahmedabad" "$file";;
3)head -2 "$file";;
4)wc "$file";;
5)echo "exiting program"; exit 0;;
*)echo "not a correct choice";;
esac
done
```

```
display_top_two() {
    echo "Top two records:"
    head -n 2 employee_file.txt
}
# Function to count the number of words in the file
count_words() {
    num_words=$(wc -w < employee_file.txt)
    echo "Number of words in the file: $num_words"
}
# Main function
main() {
    while true; do
        echo "Employee File Menu:"
        echo "a. Count the number of lines matching a given pattern"
        echo "b. Display the employees whose city is not in Ahmedabad"
        echo "c. Display the top two records"
        echo "d. Count the number of words in the file"
        echo "e. Exit"
        read -p "Enter your choice (a, b, c, d, e): " choice
        case $choice in
            a) count_matching_lines ;;
            b) display_non_ahmedabad ;;
            c) display_top_two ;;
            d) count_words ;;
            e) echo "Exiting..."; break ;;
            *) echo "Invalid choice. Please enter a valid option." ;;
        esac
    done
}
# Call the main function
main
```

45. Write a shell script to (create a menu)

a. Compare two files

b. Find the difference between two files

c. Find the common between two files

d. Split the first file by number of given lines.

Function to compare two files

```
compare_files() {
    read -p "Enter the first file name: " file1
    read -p "Enter the second file name: " file2
    if cmp -s "$file1" "$file2"; then
        echo "Files are identical."
    else
        echo "Files are different."
    fi
}
# Function to find differences between two files
find_differences() {
    read -p "Enter the first file name: " file1
    read -p "Enter the second file name: " file2
    diff "$file1" "$file2"
}
```

```
# Function to find common lines between two files
find_common() {
    read -p "Enter the first file name: " file1
    read -p "Enter the second file name: " file2
    comm -12 <(sort "$file1") <(sort "$file2")
}
# Function to split the first file by a given number of lines
split_file() {
    read -p "Enter the file name: " file
    read -p "Enter the number of lines per part: " lines
    split -l "$lines" "$file" "$file.part"
    echo "File '$file' has been split into parts with $lines each."
}
# Main function
main() {
    while true; do
        echo "File Operations Menu:"
        echo "a. Compare two files"
        echo "b. Find the difference between two files"
        echo "c. Find the common between two files"
        echo "d. Split the first file by number of given lines"
        echo "e. Exit"
        read -p "Enter your choice (a, b, c, d, e): " choice
        case $choice in
            a) compare_files ;;
            b) find_differences ;;
            c) find_common ;;
            d) split_file ;;
            e) echo "Exiting..."; break ;;
            *) echo "Invalid choice. Please enter a valid option." ;;
        esac
    done
}
# Call the main function
main
```

46. Create a text files: Fruit and vegetable and perform following operations.
a. Display how many words, lines and characters are in these files.
b. Sort both the files in increasing alphabetical order.
c. Create a directory in your home directory and move the Fruit file to it.

```
# Create Fruit and Vegetable files
echo "apple banana mango" > Fruit.txt
echo "carrot potato tomato" > Vegetable.txt
# Function to display word count, line count, and character count of a
file
display_file_info() {
    file=$1
    words=$(wc -w < "$file")
    lines=$(wc -l < "$file")
    chars=$(wc -m < "$file")
    echo "File: $file"
    echo "Words: $words"
    echo "Lines: $lines"
```

```
    echo "Characters: $chars"
}
# Function to sort files in increasing alphabetical order
sort_files() {
    sort -o Fruit.txt Fruit.txt
    sort -o Vegetable.txt Vegetable.txt
    echo "Files sorted."
}
# Function to create a directory and move the Fruit file to it
move_file() {
    mkdir -p ~/FruitDirectory
    mv Fruit.txt ~/FruitDirectory/
    echo "Fruit file moved to ~/FruitDirectory."
}
# Main function
main() {
    while true; do
        echo "Operations Menu:"
        echo "a. Display word count, line count, and character count of
files"
        echo "b. Sort both files in increasing alphabetical order"
        echo "c. Move the Fruit file to a directory in your home
directory"
        echo "d. Exit"
        read -p "Enter your choice (a, b, c, d): " choice
        case $choice in
            a)
                display_file_info Fruit.txt
                display_file_info Vegetable.txt
                ;;
            b) sort_files ;;
            c) move_file ;;
            d) echo "Exiting..."; break ;;
            *) echo "Invalid choice. Please enter a valid option." ;;
        esac
    done
}
# Call the main function
main
```

47. a) List the content of the "/var/log" directory.
    b) Create a new directory named "exam_set" in your home directory.
    c) Change your current working directory to "exam_set."
    d) Create an empty textfile named "log_file.txt" inside "exam_set".
    e) Display the last 10 lines of the "var/log/syslog" file.

```
#!/bin/bash

# Function to list contents of /var/log directory
list_var_log() {
    echo "Contents of /var/log directory:"
    ls /var/log
```

```
}

# Function to create a new directory named "exam_set" in the home
directory
create_exam_set_directory() {
    mkdir -p ~/exam_set
    echo "Directory 'exam_set' created in the home directory."
}

# Function to change the current working directory to "exam_set"
change_to_exam_set_directory() {
    cd ~/exam_set || exit
    echo "Changed directory to 'exam_set'."
}

# Function to create an empty text file named "log_file.txt" inside
"exam_set"
create_log_file() {
    touch log_file.txt
    echo "Empty text file 'log_file.txt' created inside 'exam_set'."
}

# Function to display the last 10 lines of /var/log/syslog
display_last_10_lines_syslog() {
    echo "Last 10 lines of /var/log/syslog:"
    tail -n 10 /var/log/syslog
}

# Main function
main() {
    while true; do
        echo "Operations Menu:"
        echo "a. List the contents of the /var/log directory"
        echo "b. Create a new directory named 'exam_set' in your home
directory"
        echo "c. Change your current working directory to 'exam_set'"
        echo "d. Create an empty text file named 'log_file.txt' inside
'exam_set'"
        echo "e. Display the last 10 lines of the /var/log/syslog file"
        echo "f. Exit"

        read -p "Enter your choice (a, b, c, d, e, f): " choice

        case $choice in
            a) list_var_log ;;
            b) create_exam_set_directory ;;
            c) change_to_exam_set_directory ;;
            d) create_log_file ;;
            e) display_last_10_lines_syslog ;;
            f) echo "Exiting..."; break ;;
            *) echo "Invalid choice. Please enter a valid option." ;;
        esac
    done
}
```

```
# Call the main function
main
```

48. Menu based shell script

Write a menu-based shell script name "menu_script.sh" with the following options :

1. Compare two string.
2. Join two strings.
3. Find the length of a given string.
4. Occurrence of character and words.
5. Reverse the string.

```
#!/bin/bash

# Function to compare two strings
compare_strings() {
    read -p "Enter the first string: " str1
    read -p "Enter the second string: " str2

    if [ "$str1" == "$str2" ]; then
        echo "Strings are equal."
    else
        echo "Strings are not equal."
    fi
}

# Function to join two strings
join_strings() {
    read -p "Enter the first string: " str1
    read -p "Enter the second string: " str2

    echo "Joined string: $str1$str2"
}

# Function to find the length of a string
string_length() {
    read -p "Enter the string: " str
    length=${#str}
    echo "Length of the string: $length"
}

# Function to count occurrences of characters and words
count_occurrences() {
    read -p "Enter the string: " str
    read -p "Enter the character to find its occurrence: " char
    read -p "Enter the word to find its occurrence: " word

    char_occurrences=$(grep -o "$char" <<< "$str" | wc -l)
    word_occurrences=$(grep -o "\b$word\b" <<< "$str" | wc -w)

    echo "Occurrences of character '$char': $char_occurrences"
    echo "Occurrences of word '$word': $word_occurrences"
```

occurance of word
cnt=$(echo "$str" | grep -o "$char" | wc -l)
cnt1=$(echo "$str" | grep -o "$word" | wc -l)

```
read -p "enter first string:" str1
read -p "enter sec strinf:" str2
while true
do
echo "menu:
1)compare two string
2)join two strings
3)length of string
4)occurance of word
5)reverse string
enter ur choice:"
read choice
case $choice in
1)if cmp -s "$str1" "$str2"
then
echo "compared same"
else
echo "not same"
fi;;
2) echo "joined string:$str1,$str2";;
3)length=${#str1}
echo "length of string is :$length";;
4)read -p "enter the word:" word
read -p "enter the charc:" char
cnt=$(echo "$str1" | grep -o "$word" | wc -l)
cnt1=$(echo "$str1" | grep -o "$char" | wc -l)
echo "$cnt"
echo "$cnt1";;
5)reverse=$(rev <<< "$str1")
echo "reversed is :$reverse";;
6)echo "exiting 0"; exit 0;;
*) echo "not a correct choice";;
esac
done
```

```
}

# Function to reverse a string
reverse_string() {
    read -p "Enter the string: " str
    reversed=$(rev <<< "$str")
    echo "Reversed string: $reversed"
}

# Main function
main() {
    while true; do
        echo "Menu:"
        echo "1. Compare two strings"
        echo "2. Join two strings"
        echo "3. Find the length of a given string"
        echo "4. Occurrence of character and words"
        echo "5. Reverse the string"
        echo "6. Exit"

        read -p "Enter your choice (1-6): " choice

        case $choice in
            1) compare_strings ;;
            2) join_strings ;;
            3) string_length ;;
            4) count_occurrences ;;
            5) reverse_string ;;
            6) echo "Exiting..."; break ;;
            *) echo "Invalid choice. Please enter a number between 1 and
6." ;;
        esac
    done
}

# Call the main function
main
```

49. Crud Operations on File.
    1. Create an initial file "data.txt" with some sample data.
    2. Display the content of "data.txt"
    3. Add a new record to "data.txt"
    4. Update a speciific record in "data.txt"
    5. Delete a record from "data.txt"
    6. Display the modified content of "data.txt"

```
#!/bin/bash

# Function to create an initial file with sample data
create_initial_file() {
    echo "Sample data 1" > data.txt
```

```
    echo "Sample data 2" >> data.txt
    echo "Sample data 3" >> data.txt
    echo "Initial file 'data.txt' created with sample data."
}

# Function to display the content of "data.txt"
display_content() {
    echo "Content of 'data.txt':"
    cat data.txt
}

# Function to add a new record to "data.txt"
add_record() {
    read -p "Enter the new record to add: " new_record
    echo "$new_record" >> data.txt
    echo "New record added to 'data.txt'."
}

# Function to update a specific record in "data.txt"
update_record() {
    display_content
    read -p "Enter the line number of the record to update: " line_number
    read -p "Enter the updated record: " updated_record

    sed -i "${line_number}c\\$updated_record" data.txt
    echo "Record updated in 'data.txt'."
}

# Function to delete a record from "data.txt"
delete_record() {
    display_content
    read -p "Enter the line number of the record to delete: " line_number

    sed -i "${line_number}d" data.txt
    echo "Record deleted from 'data.txt'."
}

# Main function
main() {
    while true; do
        echo "Menu:"
        echo "1. Create initial file with sample data"
        echo "2. Display content of 'data.txt'"
        echo "3. Add a new record to 'data.txt'"
        echo "4. Update a specific record in 'data.txt'"
        echo "5. Delete a record from 'data.txt'"
        echo "6. Display modified content of 'data.txt'"
        echo "7. Exit"

        read -p "Enter your choice (1-7): " choice

        case $choice in
            1) create_initial_file ;;
            2) display_content ;;
```

```
            3) add_record ;;
            4) update_record ;;
            5) delete_record ;;
            6) display_content ;;
            7) echo "Exiting..."; break ;;
            *) echo "Invalid choice. Please enter a number between 1 and
7." ;;
        esac
    done
}

# Call the main function
main
```

50. Perform the following Linux Commands
    1. List the contents of the "/etc" directory.
       ```
       ls /etc
       ```
    2. Check the freee disk space on your system.
       ```
       df -h
       ```
    3. Find and display all files in your home directory modified in the last 3 days.
       ```
       find ~/ -type f -mtime -3
       ```
       ```
       total_files=$(find "$HOME" -type f -mtime -3 | wc -l)
       echo "$total_files"
       ```
    4. Create a symbolic link named "mylink" pointing to the "/etc/passwd" file.
       ```
       ln -s /etc/passwd mylink
       ```
    5. Determine the ip address of your machine.
       ```
       ip addr show
       ```

```
while true
do
echo "menu:
1)list content of /var/log   #47 no
2)create new dir exam_set in home
3)chnage ur current dir to exam_set
4)create empty file log_file insisde exam_set
5)display last 10 lines var/log/syslog
enter ur choice:"
read choice
case $choice in
1)if mkdir -p "$HOME/var/log"
then
ls "$HOME/var/log"
fi;;
2)if mkdir -p "$HOME/exam_set"
then
echo "created successfull"
fi;;
3)if mkdir -p "$HOME/var/log/exam_set"
then
cd "$HOME/var/log/exam_set"
fi;;
4)touch "$HOME/exam_set/log_file.txt";;
5)if touch "$HOME/var/log/syslog"
then
tail -10 "$HOME/var/log/syslog"
fi;;
6)echo "exiting"; exit;;
*)echo "not a correct choice";;
esac
done
```

```
while true    #49 no
do
echo "menu :
1)add record
2)display record
3)update record
4)delete record
5)exiting
enter ur choice:"
read choice
case $choice in
1)read -p "enter the name:" name
read -p "enter the age:" age
echo "$name,$age" >> "data.txt"
echo "record added";;
2) cat "data.txt"
echo "record displayed";;
3)read -p "enter the record to updated:" name
if grep -q "^$name,"  "data.txt"
then
sed -i "/^$name,/d" "data.txt"
read -p "enter the new name:"  n_name
read -p "enter the new age:" n_age
echo "$n_name,$n_age" >> "data.txt"
echo "record updated"
fi;;
4)read -p "enter the record to delete:" name
if grep -q "^$name," "data.txt"
then
sed -i "/^$name,/d" "data.txt"
echo "record dleteed"
fi;;
5)echo "exiting"; exit 0;;
*)echo "not a correct choice";; esac done
```